

Por que PostgreSQL e MongoDB

Conexão Solidária · Hackathon 11NETT

Sumário

Resumo	1
O que o sistema faz com dados	1
PostgreSQL, para o caminho transacional	2
Unicidade imposta pelo banco, não pela aplicação	2
Incremento atômico do valor arrecadado	2
Transactional outbox	2
Dinheiro e datas com tipos corretos	3
MongoDB, para o ledger	3
O índice único é a idempotência	3
Append-only, alto volume, leitura rara	3
Esquema que vai mudar	3
Por que não um banco só	4
Só PostgreSQL	4
Só MongoDB	4
Alternativas consideradas	4
O que se paga por essa escolha	4
Verificação	5

Resumo

A Conexão Solidária usa **PostgreSQL** para o caminho transacional — usuários, campanhas, doações e a tabela de outbox — e **MongoDB** para o ledger append-only de doações processadas.

A divisão não é por tipo de dado. É por **garantia exigida**. O caminho transacional precisa de unicidade imposta pelo banco, incremento atômico e transação; o ledger precisa de escrita rápida, crescimento linear e um índice único que sirva de trava de idempotência. São requisitos diferentes o bastante para justificar motores diferentes.

Este documento liga cada escolha a uma linha concreta do sistema.

O que o sistema faz com dados

Uma doação percorre cinco passos:

1. A API grava a intenção de doação e a mensagem do evento na mesma transação
2. O outbox entrega o evento ao RabbitMQ
3. O Worker consome e tenta registrar a doação no ledger

4. Se o registro é inédito, o Worker incrementa o total da campanha
5. O painel público lê o total

O passo 1 exige transação. O passo 3 exige uma trava de unicidade barata e durável. O passo 4 exige incremento atômico. O passo 5 exige leitura rápida de um valor já calculado. É desse conjunto que as duas escolhas saem.

PostgreSQL, para o caminho transacional

Unicidade imposta pelo banco, não pela aplicação

`usuarios` tem dois índices únicos: `ix_usuarios_email` e `ix_usuarios_cpf`.

Verificar duplicidade em código antes de inserir não resolve: dois cadastros simultâneos passam os dois pela checagem e inserem os dois. A janela é pequena e real. O índice único fecha essa janela no único lugar onde a checagem e a escrita são atômicas — dentro do banco.

Este é o risco **R6** do backlog técnico, e a razão de ele estar mitigado é o motor relacional, não a aplicação.

Incremento atômico do valor arrecadado

O Worker roda com duas réplicas. Se cada uma lesse `valor_arrecadado`, somasse em memória e gravasse de volta, uma das doações desapareceria — *lost update* clássico.

O repositório nunca faz read-modify-write:

```
contexto.Campanhas
    .Where(c => c.Id == idCampanha)
    .ExecuteUpdateAsync(a => a.SetProperty(
        c => c.ValorArrecadado,
        c => c.ValorArrecadado + valor));
```

Isso vira `SET valor_arrecadado = valor_arrecadado + @valor`: uma única instrução, resolvida pelo banco sob o lock da linha. É o risco **R3**, e depende de um motor que garanta isolamento por linha.

Transactional outbox

A API precisa gravar a doação e publicar o evento sem que um dos dois possa falhar sozinho. Se o publish falhasse depois do commit, a doação existiria e nunca seria processada: o valor nunca subiria, e nenhum erro apareceria.

O MassTransit resolve isso gravando a mensagem numa tabela de outbox **dentro da mesma transação** da doação, e entregando depois:

```
configurador.AddEntityFrameworkOutbox<AppDbContext>(outbox =>
{
    outbox.UsePostgres();
    outbox.UseBusOutbox();
});
```

O padrão só existe porque há uma transação que abrange as duas escritas. Sem banco transacional, não há outbox — e o risco **R1** volta.

Dinheiro e datas com tipos corretos

`valor_arrecadado` e `meta_financeira` são `numeric`, não ponto flutuante: não há erro de arredondamento acumulado. As datas são `timestampz`, casadas com `DateTimeOffset` na aplicação, o que elimina a classe de bug em que um container em UTC compara contra um horário local (risco **R5**).

MongoDB, para o ledger

O índice único é a idempotência

RabbitMQ entrega *at-least-once*. Se o Worker processa, incrementa e o ACK se perde, a mensagem volta — e somar de novo inflaria o total arrecadado.

A trava é a coleção `doacoes_processadas`, com índice único em `idDoacao`. O Worker não consulta antes de inserir; ele tenta inserir e deixa o banco decidir:

```
try
{
    await _colecacao.InsertOneAsync(registro, cancellationToken: ct);
    return true; // inédita: pode somar
}
catch (MongoWriteException e)
    when (e.WriteError?.Code == 11000) // chave duplicada
{
    return false; // redelivery: descarta sem somar
}
```

Consultar-e-inserir teria a mesma janela de corrida do cadastro por e-mail. Tentar inserir e tratar a violação não tem janela nenhuma. É o risco **R2**.

Append-only, alto volume, leitura rara

O ledger só cresce. Nenhum documento é atualizado ou apagado, e ele é lido apenas para auditoria e reconciliação — nunca no caminho da requisição do doador, e nunca para montar o painel público.

Essa é uma carga em que o modelo documental rende: escrita direta sem relações a manter, sem joins, e crescimento linear sem competir por lock com o caminho transacional.

Esquema que vai mudar

O registro hoje guarda doação, campanha, doador, valor e instante de processamento. Auditoria é o tipo de coisa a que se acrescenta campo — origem do pagamento, identificador de conciliação, resultado de antifraude — sem que os documentos antigos precisem ser reescritos. Em tabela relacional, cada acréscimo seria uma migration sobre uma tabela que só cresce.

O valor é gravado como `Decimal128`, e não como `double`: o mesmo cuidado com dinheiro do lado relacional.

Por que não um banco só

Só PostgreSQL

Tecnicamente possível: o ledger seria uma tabela com índice único em `id_doacao`, e a idempotência funcionaria igual.

O que se perde é o **isolamento entre as duas cargas**. A tabela de auditoria cresce indefinidamente, e passa a disputar cache, `VACUUM` e manutenção de índice com as tabelas que estão no caminho da requisição. A carga que só precisa durar degrada a carga que precisa responder rápido.

Separar também deixa explícito no desenho quem é fonte da verdade transacional e quem é histórico — o que, na prática, evita que alguém escreva um relatório pesado direto contra o banco das campanhas.

Só MongoDB

Descartado pelo caminho transacional. Transação multi-documento existe no MongoDB, mas exige replica set e custa mais caro que a transação local do Postgres, que é o caso de uso mais frequente do sistema. O outbox do MassTransit com EF Core, usado aqui, também é uma integração de banco relacional.

Trocar um banco que dá transação de graça por um em que ela é exceção, para então reimplementar as garantias, seria andar para trás.

Alternativas consideradas

Alternativa	Por que foi descartada
Redis para idempotência	É a trava mais barata que existe, mas volátil e com TTL. O ledger precisa durar para auditoria, não por alguns minutos. Guardar a trava em um lugar e o histórico em outro criaria dois pontos que podem divergir.
Cassandra para o ledger	Escala escrita melhor que o MongoDB, e o custo operacional é muito maior. O volume deste sistema não chega perto de justificar.
Event sourcing completo	Elegante para o domínio, mas trocava uma coluna materializada e um <code>UPDATE</code> atômico por projeções e replay. Complexidade sem retorno num MVP de cinco casos de uso.
SQL Server no lugar do PostgreSQL	Atenderia igual. PostgreSQL foi escolhido por rodar sem licença em container e no cluster local — o professor precisa subir o ambiente na máquina dele.
Um banco só, relacional	Ver acima: perde o isolamento entre a carga transacional e o histórico que só cresce.

O que se paga por essa escolha

Honestidade sobre o custo:

- **Duas dependências de disponibilidade.** O `/health/ready` checa os dois, e o Worker precisa dos dois para processar uma doação.
- **Não há transação entre eles.** O Worker grava no Mongo e depois incrementa no Postgres. Se cair entre as duas escritas, a doação fica registrada no ledger sem ter sido somada — e a redelivery será descartada como duplicata. A ordem foi escolhida deliberadamente: esse cenário perde uma soma, enquanto a ordem inversa somaria duas vezes. Perder é reconciliável; somar a mais, não.
- **Reconciliação é responsabilidade nossa.** Conferir o ledger contra o total materializado é um procedimento operacional, não uma garantia do banco.

Verificação

Não é argumento teórico. Com o Worker em duas réplicas no cluster:

- 30 doações simultâneas de R\$ 10,00 resultaram em exatamente **R\$ 300,00** — o incremento atômico segura concorrência real
- Redeliveries aparecem na métrica `doacoes_duplicadas_descartadas_total`, e o total arrecadado não se move quando elas chegam
- O ledger, consultado direto no pod do MongoDB, tem um documento por doação processada, com o valor em `Decimal128`

Documento gerado de `docs/escolha-dos-bancos.md`. Diagrama da arquitetura em `docs/diagramas/arquitetura`.